



Міністерство освіти і науки України
Національний технічний університет України “Київський політехнічний
інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №4
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛони «SINGLETON», «ITERATOR», «PROXY», «STATE»,
«STRATEGY»»**
Музичний програвач

Виконав:
Студент групи ІА-23
Волошин В. В.

Перевірив:
Мякий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:.....	3
Хід роботи.....	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми.....	3
2. Реалізувати один з розглянутих шаблонів за обраною темою.....	4
Висновки:.....	6
Код проекту:.....	6

Тема:

ШАБЛОНИ «SINGLETON», «ITERATOR», «PROXY», «STATE», «STRATEGY»

Мета:

Ознайомитися з основними шаблонами проектування, такими як «Singleton», «Iterator», «Proxy», «State» та «Strategy», дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Хід роботи

1. Реалізувати не менше 3-х класів відповідно до обраної теми

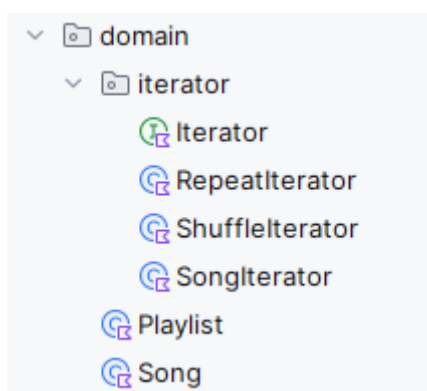


Рис. 1 — Структура проекту

У процесі виконання лабораторної роботи було створено шість класів, які забезпечують реалізацію функціоналу музичного програвача (рис. 1). Наведемо детальний опис кожного з цих класів:

1. Song

Клас відповідає за зберігання інформації про окрему пісню. Містить у собі такі властивості, як унікальний ідентифікатор (id), назва пісні (name), і URL-адреса для доступу до файлу пісні (songUrl). Використовується для зберігання даних у плейлисті.

2. Playlist

Основний клас для управління колекцією пісень. Реалізує функціонал для додавання, зберігання та видалення пісень із плейлиста. Крім того, надає метод `createIterator()`, який дозволяє створити конкретний ітератор (послідовний, перемішаний або повторюваний) для обходу списку пісень залежно від обраного режиму.

3. SongIterator

Базовий клас-ітератор для послідовного обходу колекції пісень. Забезпечує методи `hasNext()` для перевірки наявності наступного елемента та `next()` для доступу до наступної пісні у плейлисті. Використовується як основа для створення спеціалізованих ітераторів.

4. ShuffleIterator

Наслідує `SongIterator` та реалізує алгоритм для обходу пісень у випадковому порядку. При створенні об'єкта даного класу список пісень перемішується, а доступ до елементів здійснюється у випадковій послідовності. Застосовується для реалізації режиму відтворення "перемішування" (`shuffle`).

5. RepeatIterator

Також наслідує `SongIterator`, але забезпечує зациклений обхід плейлиста. Після досягнення кінця списку відтворення повертається на початок. Цей ітератор використовується для реалізації режиму "повторення" (`repeat`).

6. Iterator

Інтерфейс, який визначає загальні методи для всіх ітераторів (`hasNext()` і `next()`). Він дозволяє уніфікувати доступ до елементів плейлиста незалежно від конкретної реалізації ітератора. Усі ітератори (послідовний, перемішаний, повторюваний) реалізують цей інтерфейс.

Ця структура класів забезпечує модульність і зручність роботи зі списком пісень, а також спрощує розширення функціоналу програвача у майбутньому.

2. Реалізувати один з розглянутих шаблонів за обраною темою

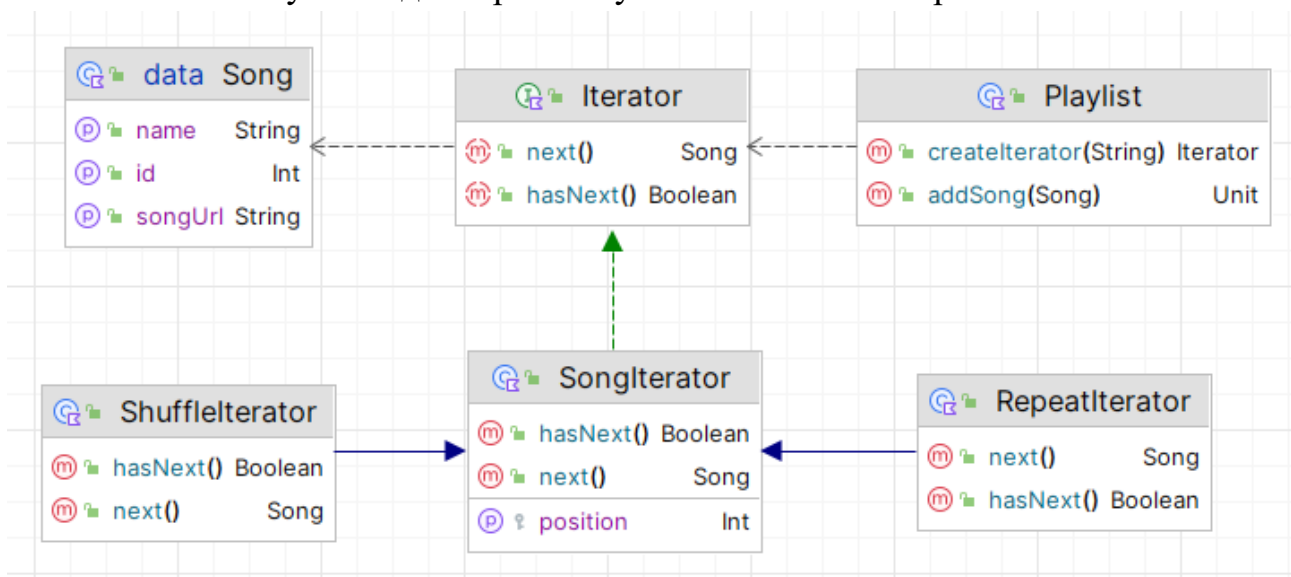


Рис. 2 — Діаграма класів

У рамках проєкту музичного програвача патерн Iterator було реалізовано для забезпечення гнучкого та інкапсульованого доступу до списків пісень (плейлистів). Його використання дозволило створити уніфікований механізм обходу колекцій пісень, незалежно від їхньої внутрішньої структури чи способу зберігання.

Реалізація

Основна ідея полягала у створенні ітераторів, які дозволяють працювати зі списком пісень у різних режимах: послідовно, у випадковому порядку (shuffle) або зациклено (repeat).

- Плейлист виступає колекцією пісень, яка підтримує додавання, редагування та видалення треків. Він створює відповідний ітератор залежно від обраного режиму.
- Ітератори інкапсують логіку обходу плейлисту, приховуючи деталі реалізації списку пісень від основної програми.

Проблеми, які вирішує патерн

- Інкапсуляція доступу до колекцій. Завдяки ітераторам користувачі можуть працювати зі списками пісень через єдиний інтерфейс без необхідності знати їхню внутрішню структуру (масив, список, база даних тощо).
- Уніфікованість обходу. Незалежно від режиму (послідовний, перемішаний, повторюваний), методи доступу (`hasNext()` та `next()`) залишаються однаковими, що спрощує взаємодію з колекцією.
- Легкість додавання нових режимів обходу. Завдяки базовому інтерфейсу Iterator і можливості створення підкласів, легко додавати нові режими (наприклад, фільтрація за жанром або виконавцем), не змінюючи існуючий код.
- Покращення читабельності та підтримки коду. Відокремлення логіки обходу колекції від основного функціоналу програвача дозволяє створювати код, який легше підтримувати, тестувати та модифікувати.

Переваги використання

- Модульність: Патерн забезпечує чітке розмежування відповідальності між зберіганням даних (плейлистом) і їхнім обходом (ітераторами).
- Гнучкість: Додаючи новий тип ітератора, можна розширити функціонал програвача без змін в існуючій логіці.

- Повторне використання: Ітератори можуть бути повторно використані для інших колекцій або проєктів із подібною структурою.
- Уніфікований доступ: Програвач працює з усіма режимами обходу через однакові методи, що спрощує його розробку та використання.

Висновки:

У ході лабораторної роботи було реалізовано патерн Iterator для роботи з плейлистами музичного програвача. Використання цього шаблону дозволило спростити доступ до колекцій пісень і підвищити гнучкість та модульність коду, що значно полегшує його підтримку та розширення.

Код проєкту:

<https://github.com/vv0loshyn/TRPZ-labs>